RESIDÊNCIA EM MICROELETRÔNICA — POLO UFCG

Design Verification

Verification Plan

Fase 2 – PARTE II

Projeto Integrador de Design Verification

planejamento da verificação do sistema de topo com AES

Instrutor e Autor:
Gabriel Villanova

Aluno:
Alessandro Lucas de Arruda Santos

Documento:
Verification Plan – Fase 2

CONFIDENCIAL
V1.0.0
Setembro/2026

Sumário

1 Introdução e fluxo de planejamento
2 DUT, interfaces e escopo
3 Features e requisitos verificáveis
4 Arquitetura, layers e fases de bring-up
5 Plano de estímulos
6 Plano de checagem
7 Plano de cobertura
8 Testes e cenários
9 Rastreabilidade e critérios de closure
10 Referências

“Hard work always pays off.”

1 Introdução e fluxo de planejamento

O Verification Plan define a estratégia de verificação para o sistema de topo da Fase 2. Ele deve transformar a especificação do DUT em features verificáveis, estímulos, mecanismos de observação e checagem, cobertura, testes, rastreabilidade e critérios objetivos de fechamento.

1.1 Objetivos do VPLAN

- Registrar o escopo funcional a ser verificado e o que está explicitamente fora de escopo.
- Derivar mecanismos de verificação a partir das features do DUT.
- Planejar estímulos, checkers, reference models, assertions e cobertura antes do closure.
- Manter rastreabilidade entre feature → cenário → checker → coverage → teste → evidência.
- Definir quando a verificação poderá ser considerada tecnicamente concluída.

1.2 Fluxo de raciocínio esperado

Etapa	Pergunta que o aluno deve responder	Saída esperada
1. Especificação	O que o sistema promete fazer?	Requisitos e interfaces compreendidos.
2. Features	Quais comportamentos precisam de evidência?	Lista de features com IDs únicos.
3. DITL / cenários	Como o sistema é usado do reset ao processamento?	Fluxos realistas e corner cases.
4. Arquitetura	O que preciso gerar, observar e comparar?	VIPs, monitors, scoreboard/refmodel, assertions.
5. Mecanismos	Como cada feature será estimulada e medida?	Stimuli, checkers, coverage e testes.
6. Regressão	Como repetir e ampliar o espaço exercitado?	Suíte automatizada com seeds e logs.
7. Closure	O que ainda falta e por quê?	Gaps fechados ou waivers justificados.

2 DUT, interfaces e escopo

2.1 Visão funcional do DUT

O MiniSoC AES é um sistema que integra controle via SPI, geração de clock via PLL (modelo RTL), um engine criptográfico AES-128 e uma memória compartilhada de 1024x32 bits. O fluxo funcional envolve a inicialização com reset global, aguardando o lock do PLL e a liberação do clock do sistema. A memória, inicialmente controlada pela interface externa, recebe os dados de entrada. Após configurar chaves, vetores de inicialização (IV), modo (ECB, CBC, PCBC, CFB, OFB, CTR) e número de blocos (1 a 256) via SPI (máximo 25MHz, Mode 0), a posse da memória (ownership) é transferida ao AES (MEM_SEL_AES=1). O processamento ocorre in-place e, ao finalizar (DONE=1), a memória é devolvida ao domínio externo para leitura dos resultados.

2.2 Interfaces do DUT

Interface	Função no sistema	O que deve ser verificado	Observabilidade
RESET (rst_n)	IInicialização global (ativo em nível baixo)	Assert/deassert, reset durante operação e inicialização do SPI independente do PLL lock.	Sinais de estado, monitors e assertions.
CLOCK / PLL	Clock 2GHz (clk) e PLL system clock	Liberação do clock interno apenas após sinal de locked e configuração dos divisores (/1, /2, /4, /8).	Clock monitor, locked e efeitos no sistema.
SPI (spi_sclk, cs_n, mosi, miso)	Controle/Status (max 25MHz, Mode 0)	Operações assíncronas ao sys_clk. Respeito ao protocolo MSB-first e leitura em duas transações.	SPI monitor + estado funcional observado.
MEMORY DATA (mem_clk, mem_i_*, mem_o_*)	Interface de dados externa (até 200MHz)	Leitura/escrita de 1024 posições de 32 bits. Exclusão mútua de acesso imposta pelo multiplexador de ownership.	Monitor transacional + scoreboard.
AES datapath	Processamento criptográfico interno	Suporte aos 6 modos exigidos, rotinas decrypt/encrypt de 1 a 256 blocos e gravação in-place sem vazamento de dados.	Reference model + comparação fim a fim.

2.3 Dentro e fora de escopo

Item	Status	Justificativa / observação
Funcionalidade digital top-level	IN	Obrigatória. Todas as interações do RTL devem ser testadas.
AES funcional	IN	Comparação end-to-end com AES Reference Model via Scoreboard.
Arbitragem da Memória (Ownership)	IN	Garantir a exclusão mútua exigida pela spec de controle e memória compartilhada.
Comportamento analógico real do PLL	OUT / MOCK	O bloco minisoc_pll_mock.sv simula um PLL digitalmente, injetando atrasos pseudoaleatórios para o clock lock.
Timing físico pós-layout	OUT	Não faz parte do objetivo desta fase.

3 Features e requisitos verificáveis

ID	Feature	Descrição comportamental	Prioridade
F01	Reset Global e Domínio Always-On	O DUT deve inicializar a lógica de controle sob `rst_n=0`, manter o SPI acessível pelo clock de 2 GHz mesmo sem lock do PLL e preservar o conteúdo da memória física após reset	Alta
F02	Controle de PLL e Liberação de Clock	O PLL mock deve dividir o clock de 2 GHz por `/1`, `/2`, `/4` ou `/8` quando habilitado (`PLL_ENABLE=1`) e só liberar `sys_clk` ao domínio AES após indicar `PLL_LOCKED=1`	Alta
F03	Protocolo SPI e Banco de Registradores	O front-end SPI (Mode 0, MSB-first, até 25 MHz) deve processar escritas em 1 transação e leituras em 2 transações via CDC, protegendo registradores *Read-Only* e ignorando endereços não mapeados.	Alta
F04	Arbitragem e Exclusão Mútua da Memória	O acesso à memória (1024×32 bits) deve respeitar estritamente MEM_SEL_AES (0=externo até 200 MHz; 1=AES), impedindo interferência entre domínios e validando o alinhamento *big-endian* de 4 words por bloco	Alta
F05	Datapath Criptográfico AES-128 In-Place	O engine deve cifrar e decifrar blocos de 128 bits nos 6 modos suportados (ECB, CBC, PCBC, CFB, OFB, CTR), sobrescrevendo apenas a região configurada da memória.	Alta
F06	Processamento Multi-Bloco e Status	Ao receber `START=1` válido, o AES deve capturar a configuração, processar de 1 a 256 blocos incrementando `BLOCKS_DONE` e manter `DONE=1` (*sticky*) até um novo `START` válido.	Alta
F07	Detecção de Erros de Configuração	Configurações fora do intervalo válido (`NUM_BLOCKS` fora de `1..256` ou `MODE` em `110`/`111`) devem elevar `ERROR=1` no início da operação e encerrar sem alterar a memória.	Alta
F08	Robustez a Eventos Assíncronos e Runtime	O DUT deve suportar reset em operação, ignorar escritas concorrentes ou novos pulsos de `START` durante `BUSY=1` (graças ao *snapshot* interno) e descartar frames SPI incompletos	Média

3.1 Day-in-the-Life (DITL) / cenários de uso

Escreva de 2 a 5 fluxos realistas que atravessem múltiplas features. Um DITL não substitui os testes; ele ajuda a descobrir interações que testes isolados podem perder.

ID	Fluxo resumido	Features relacionadas
DITL-01	**Fluxo Nominal End-to-End (Cifragem + Decifragem In-Place):** Reset → Grava blocos na memória (MEM_SEL_AES=0) → Habilita PLL e aguarda PLL_LOCKED=1 → Configura KEY, IV/Counter, MODE e NUM_BLOCKS via SPI → Passa posse ao AES (MEM_SEL_AES=1) e emite START=1 → Aguarda DONE=1 → Reconfigura para DECRYPT=1 e roda novamente *in-place* → Devolve memória (MEM_SEL_AES=0) e verifica retorno ao *plaintext* original.	F01, F02, F03, F04, F05, F06
DITL-02	**Rejeição de Configuração Inválida e Recuperação:** Configura MODE=110/111 ou NUM_BLOCKS=0/300 → Emite START=1 → Verifica ERROR=1 e BLOCKS_DONE=0 → Corrige parâmetros via SPI, emite novo START=1 e valida limpeza do erro e execução normal.	F03, F06, F07
DITL-03	**Tentativa de Intrusão Externa e Sobrescrita em Runtime:** Durante processamento longo (NUM_BLOCKS=256, BUSY=1), injeta escritas na interface MEMORY DATA, altera KEY/IV via SPI e envia novo START=1 → Verifica que o resultado final do AES usa o *snapshot* original e não sofreu corrupção externa.	F04, F05, F06, F08
DITL-04	**Reset em Operação (*Mid-Flight Reset*) e Persistência de RAM:** Inicia processamento AES multi-bloco → Aplica rst_n=0 no meio da execução → Verifica queda de BUSY e PLL_LOCKED → Lê a memória externamente para checar persistência física e reinicia o fluxo completo do zero	F01, F02, F04, F08

“Hard work always pays off.”

4 Arquitetura, layers e fases de bring-up

4.1 Verification layers

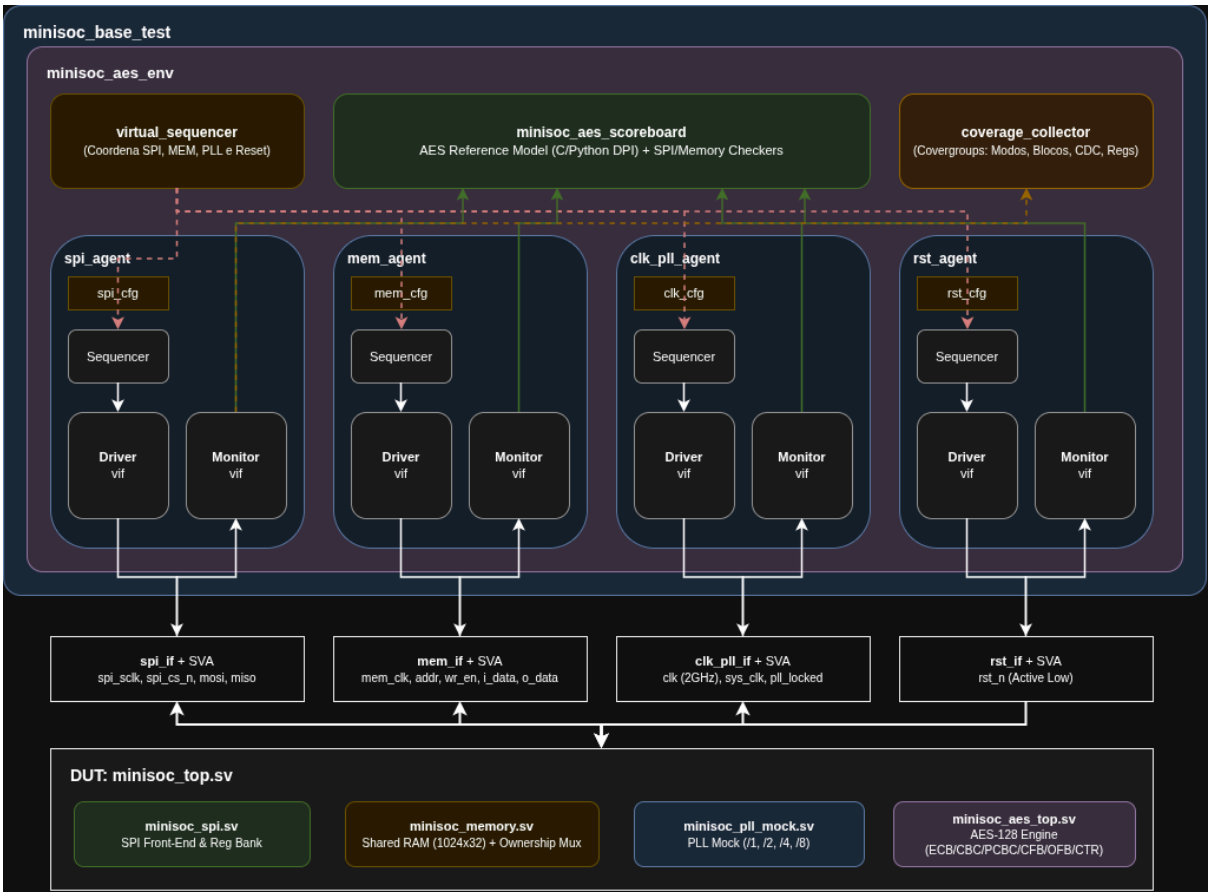
Layer	Responsabilidade	Exemplos no projeto
L0 Baseline	Compilar/elaborar, resetar e conectar DUT.	tb_top, interfaces, config_db, smoke test.
L1 Stimulus/Monitor	Gerar e observar transações.	VIP CLOCK, RESET, SPI, MEMORY DATA.
L2 Checkers	Determinar correto/incorreto.	Scoreboard, AES refmodel, SVA.
L3 Coverage	Medir o espaço funcional exercitado.	Covergroups, crosses, assertion coverage.
L4 Scenarios	Compor fluxos realistas multi-interface.	Sequences e virtual sequences.

4.2 Bring-up phases

Fase	Capacidade obtida	Critério de pronto
P0	Baseline do TB	Compila e roda smoke test.
P1	Clock + reset	DUT inicializa de forma observável e repetível.
P2	SPI	Configuração básica é aplicada e monitorada.
P3	MEMORY DATA	Transações de leitura/escrita são geradas e reconstruídas por monitor.
P4	AES self-checking	Fluxo end-to-end compara DUT vs. modelo.
P5	Coverage + regression	Cobertura coletada e suíte automatizada.
P6	Closure	Gaps tratados, regressão limpa, documentação fechada.

4.3 Arquitetura do testbench

O ambiente UVM (`minisoc_aes_env`) é composto por quatro agentes ativos principais que estimulam as interfaces do DUT: `spi_agent` (configuração de chaves, modos e controle), `mem_agent` (leitura e escrita na interface externa de memória), `clk_pll_agent` (geração do clock de referência e monitoramento do estado de lock) e `rst_agent` (controle de reset). Um `virtual_sequencer` coordena o fluxo de estímulos de alto nível, sincronizando a gravação inicial na memória, a configuração via SPI e a troca de ownership (`MEM_SEL_AES`). Os monitores de SPI e Memória enviam transações para o `minisoc_aes_scoreboard`. O Scoreboard utiliza um *Reference Model* em C/Python para calcular os resultados criptográficos esperados e compará-los *bit a bit* com os dados extraídos da memória após a flag `DONE=1` ser acionada.



5 Plano de estímulos

COMO DEFINIR ESTÍMULOS

Descreva o espaço que pode ser aplicado ao DUT, não a sequence específica que o gera. Inclua valores legais, limites, dependências, temporização e interações com reset/backpressure.

ID	Interface	Parâmetro / operação	Valores / condições	Objetivo
ST-RST-01	RESET	Reset de inicialização	Assert/deassert de `rst_n` com `PLL_ENABLE=0` e `PLL_ENABLE=1`.	Validar estado pós-reset dos registradores e bootstrap always-on
ST-RST-02	RESET	Reset em runtime (*Mid-Flight*)	Pulso `rst_n=0` durante frame SPI (ex.: bit 20/40) e durante `AES_STATUS.BUSY=1`	Validar aborto determinístico e retenção dos dados na memória física.
ST-PLL-01	SPI / PLL	Seleção de divisores e Lock	`PLL_DIV_SEL` em `00` (2 GHz), `01` (1 GHz), `10` (500 MHz) e `11` (250 MHz).	Exercitar o modelo do PLL e o CDC em todas as razões de frequência
ST-PLL-02	SPI / PLL	Distúrbios de PLL	`START=1` antes de `PLL_LOCKED=1`; queda de `PLL_ENABLE` ou troca de `PLL_DIV_SEL` com `BUSY=1`	Testar dependências de clock/reset interno e cantos de bloqueio do `sys_clk`
ST-SPI-01	SPI	Transações R/W legais	Escritas (`RWb=0`) e Leituras em 2 fases (`RWb=1` + `0x3F` NOP) em `0x00..0x0F` até 25 MHz	Programar e monitorar todo o mapa de registradores no limite de frequência.
ST-SPI-02	SPI	Acessos ilegais e RO	Escritas em registradores RO (`0x0C`, `0x0D`, `0x0F`, `0x3F`), endereços `0x10..0x3E` e bit `RSV=1`	Garantir proteção de registradores de status/versão e descarte de lixo
ST-SPI-03	SPI	Estresse de protocolo	Frames abortados (`spi_cs_n` sobe antes de 40 bits) e leituras consecutivas sem NOP	Validar robustez do sincronizador CDC de toggle e máquina serial SPI
ST-MEM-01	MEMORY	Escrita/Leitura nominal	Transações síncronas em `mem_clk` (até 200 MHz) nos endereços `0..1023` com `MEM_SEL_AES=0`.	Carregar blocos de entrada (1 a 256) e descarregar resultados criptografados.
ST-MEM-02	MEMORY	Concorrência e Ownership	Acessos externos (`mem_i_wr_en=1`) com `MEM_SEL_AES=1`; troca de `MEM_SEL_AES` durante `BUSY=1`	Verificar exclusão mútua estrita e isolamento do mux de memória
ST-AES-01	SPI / AES	Modos e Direções Válidas	`MODE` de `000` a `101` (ECB, CBC, PCBC, CFB, OFB, CTR) × `DECRYPT` (`0` e `1`).	Exercitar o datapath criptográfico (*cipher* e *inverse cipher*) em todos os modos
ST-AES-02	SPI / AES	Tamanhos de Bloco e Limites	`NUM_BLOCKS` em `1` (mínimo), `2..255` (intermediários) e `256` (memória cheia - 1024 words).	Validar controle de laço multi-bloco, processamento *in-place* e ausência de overflow
ST-AES-03	SPI / AES	Parâmetros Ilegais (Erro)	`NUM_BLOCKS = 0`, `257`, `0xFFFF`; `MODE = 110` e `111` acionando `START=1`	Forçar a detecção de erro imediato (`ERROR=1`) sem alterar a memória.
ST-AES-04	SPI / AES	Vetores Extremos e Rollover	Chaves/IV/Dados `0x0`, `0xFF..FF`, FIPS-197; `COUNTER` em `0xFFFF_FFFF` e `64'hFFFF_FFFF_FFFF_FFFF`; escrita SPI e duplo `START` com `BUSY=1`.	Validar aritmética do AES, *rollover* de contador no modo CTR e *snapshot* de registradores no `START`.

6 Plano de checagem

ID	Checker / modelo	Entrada observada	Regra de decisão
CHK-01	AES Golden Reference Model (Scoreboard E2E)	Configurações SPI capturadas no START + blocos lidos da memória antes e após DONE=1	O resultado *in-place* nos blocos 0 a $NUM_BLOCKS - 1$ deve ser idêntico bit a bit ao modelo de referência (C/Python), respeitando a ordenação *big-endian* (Word 0=[127:96]..Word 3=[31:0]).
CHK-02	Memory Boundary & Isolation Checker	Conteúdo total da RAM (1024×32) antes e depois do processamento ou durante MEM_SEL_AES=1	As posições de memória de $NUM_BLOCKS \times 4$ até 1023 não podem sofrer alteração pelo AES, e tentativas de escrita externa com MEM_SEL_AES=1 devem ser descartadas
CHK-03	SPI Register Bank & RO Checker	Transações no monitor SPI (MOSI/MISO) e valores lidos em 0x00..0x0F.	Leituras de registradores R/W devem retornar o último valor escrito; escritas em RO (0x0C, 0x0D, 0x0F) ou frames abortados (<40 bits) não podem alterar o estado.
CHK-04	Invalid Config & Error Checker	AES_CONTROL, NUM_BLOCKS e leitura de AES_STATUS após START=1.	Se MODE $\in \{6, 7\}$ ou NUM_BLOCKS $\notin [1..256]$, AES_STATUS.ERROR deve ir a 1, BLOCKS_DONE deve ser 0 e a memória deve permanecer inalterada
CHK-05	Status & Progress Tracker	Leituras de AES_STATUS (BUSY, DONE, BLOCKS_DONE) durante e após a execução.	BLOCKS_DONE deve incrementar monotonicamente até NUM_BLOCKS; DONE deve permanecer em 1 (*sticky*) após o fim até que um novo START válido seja aceito

6.1 Assertions planejadas

ID	Propriedade	Quando vale	Falha significa
A-01	MEM_MUX_EXCLUSIVITY	Sempre (clk / mem_clk).	Quando MEM_SEL_AES == 1, qualquer pulso em mem_i_wr_en propagou escrita para a memória física, violando REQ-MEM-03.
A-02	SYS_CLK_GATED_UNTIL_LOCK	Enquanto PLL_LOCKED == 0	O clock do sistema (sys_clk) apresentou transições antes do PLL atingir o estado de lock, violando REQ-PLL-01
A-03	SPI_MODE0_AND_FRAME_LEN	Durante spi_cs_n == 0	Ocorreu atualização de registrador interno com frame diferente de 40 bordas de spi_sclk ou violação de amostragem Mode 0
A-04	BUSY_DONE_ERROR_MUTEX	Domínio de controle / sys_clk	Os sinais BUSY e DONE (ou BUSY e ERROR) ficaram ativos simultaneamente em AES_STATUS após o início da operação.

ID	Propriedade	Quando vale	Falha significa
A-05	DONE_STICKY_BEHAVIOR	Após transição `DONE: 0 -> 1` sob `rst_n == 1`.	O sinal `DONE` caiu para `0` espontaneamente sem que um novo pulso de `START` válido ou reset global tenha ocorrido.
A-06	ERROR_IMMEDIATE_ABORT	No ciclo de avaliação de `START` com configuração ilegal.	Um comando `START` com `MODE > 5` ou `NUM_BLOCKS` fora de `1..256` elevou `BUSY` ou acionou escrita na memória em vez de elevar `ERROR=1` imediatamente.

7 Plano de cobertura

Coverage ID	Comportamento medido	Bins / cross planejado	Feature
COV-01	Modos de Operação AES (`MODE`)	Bins válidos: `ECB(0)`, `CBC(1)`, `PCBC(2)`, `CFB(3)`, `OFB(4)`, `CTR(5)`. Bins de erro: `RSV6(6)`, `RSV7(7)`	F05, F07
COV-02	Quantidade de Blocos (`NUM_BLOCKS`)	Bins válidos: `min(1)`, `low[2:15]`, `mid[16:255]`, `max(256)`. Bins de erro: `zero(0)`, `overflow[257:$]`	F06, F07
COV-03	Cross: Modo × Direção × Tamanho	Cross entre COV-01 (modos válidos) × DECRYPT {0, 1} × COV-02 (min, mid, max)	F05, F06
COV-04	Configuração do PLL e CDC	Bins de PLL_DIV_SEL: /1(00), /2(01), /4(10), /8(11) × Operação AES ativa	F02, F03
COV-05	Mapa de Registradores SPI	Endereços 0x00..0x0F e 0x3F × Operação (READ, WRITE), incluindo tentativa de escrita em RO (0x0C, 0x0D, 0x0F) e endereços não mapeados	F03
COV-06	Endereçamento e Posse da Memória	Endereços limite (0x000, [0x001:0x3FE], 0x3FF) × mem_i_wr_en {0, 1} × MEM_SEL_AES {0, 1}	F04
COV-07	Vetores de Dados, Chaves e CTR Rollover	Classes de vetores (`all_zeros`, `all_ones`, `random`, `nist_fips197`) e transição de carry/rollover em `COUNTER_LO`/`COUNTER_HI` no modo CTR	F05
COV-08	Eventos de Exceção e Reset	Reset por contexto (`idle`, `spi_active`, `aes_busy`), `START` duplo com `BUSY=1` e `START` com `PLL_LOCKED=0`	F01, F08

7.1 Code coverage e análise de holes

Registre line/block/expression/toggle/FSM quando disponíveis. Holes devem ser classificados: falta de estímulo, código inacessível, comportamento fora de escopo, limitação do modelo ou possível bug. Exclusões/refinements precisam de justificativa e evidência.

8 Testes e cenários

COMO ESCREVER UM TESTE NO VPLAN

O teste é uma composição de estímulos e checkers para cobrir uma intenção verificável. Não replique toda a implementação UVM aqui. Registre objetivo, fluxo, variações/seeds, checkers ativos e coverage esperado.

Test ID	Objetivo / fluxo	Checkers	Coverage principal
T-SMOKE	Reset global, bootstrap SPI, habilita PLL (`/1`), carrega 1 bloco na memória, executa AES-ECB Encrypt e confere resultado	CHK-01, CHK-03, A-02	COV-01 (ECB), COV-02 (min)
T-SPI-REGS	Varredura de leitura/escrita em todos os registradores (`0x00..0x0F`) antes do lock do PLL, escrita em RO (`0x0C`, `0x0D`, `0x0F`), endereços não mapeados e frames abortados.	CHK-03, A-03	COV-05
T-AES-MULTI-MODE	Executa cifragem e decifragem nos 6 modos válidos (ECB, CBC, PCBC, CFB, OFB, CTR) variando `NUM_BLOCKS` entre `1`, valores intermediários e `256`	CHK-01, CHK-02, CHK-05, A-04, A-05	COV-01, COV-02, COV-03
T-AES-ROUNDTRIP	Cifra N blocos *in-place* (DECRYPT=0) e, sem recarregar a memória externa, decifra os mesmos N blocos *in-place* (DECRYPT=1), validando o retorno exato ao *plaintext* (especialmente em CFB, OFB e CTR).	CHK-01, CHK-05	COV-03
T-CTR-ROLLOVER	Executa modo CTR (`101`) com múltiplos blocos inicializando o contador em `32'hFFFF_FFFE` e `64'hFFFF_FFFF_FFFF_FFFE`, além de vetores extremos (`0x0`, `0xFF..FF`).	CHK-01	COV-07
T-BOUNDARY-ERR	Configura `MODE` reservado (`110`, `111`) e `NUM_BLOCKS` inválido (`0`, `257`, `0xFFFF`), emite `START=1`, valida `ERROR=1` imediato e testa recuperação em seguida	CHK-02, CHK-04, A-06	COV-01, COV-02 (bins de erro)
T-MEM-ISOLATION	Preenche toda a RAM (1024 words), roda AES parcial ($N < 256$) para checar ausência de overflow *in-place* e injeta escritas externas (mem_i_wr_en=1) enquanto MEM_SEL_AES=1	CHK-01, CHK-02, A-01	COV-06
T-SNAPSHOT-BUSY	Dispara AES para 256 blocos e, enquanto `BUSY=1`, sobrescreve `KEY`, `IV`, `MODE`, `NUM_BLOCKS` via SPI e envia pulsos extras de `START=1`	CHK-01, CHK-05	COV-08
T-PLL-CDC-STRESS	Executa fluxos completos alternando os 4 divisores do PLL (`/1`, `/2`, `/4`, `/8`) com `spi_sclk` em 25 MHz e testa envio de `START=1` antes de `PLL_LOCKED=1`.	CHK-01, CHK-03, A-02	COV-04, COV-08

Test ID	Objetivo / fluxo	Checkers	Coverage principal
T-RST-RUNTIME	Aplica `rst_n=0` assíncrono durante transações SPI e durante `BUSY=1` do AES, verificando reinicialização de controle, preservação da memória física e recuperação pós-reset.	CHK-02, CHK-03, A-02, A-05	COV-08

9 Rastreabilidade e critérios de closure

Feature	Stimuli / Testes	Checker	Coverage	Status final
F01 (Reset & Always-On)	ST-RST-01, ST-RST-02; T-SMOKE, T-SPI-REGS, T-RST-RUNTIME	CHK-02, CHK-03, A-02	COV-05, COV-08	[OPEN]
F02 (PLL & Clock Lock)	ST-PLL-01, ST-PLL-02; T-SMOKE, T-PLL-CDC-STRESS	CHK-01, A-02	COV-04, COV-08	[OPEN]
F03 (SPI & Registradores)	ST-SPI-01, ST-SPI-02, ST-SPI-03; T-SPI-REGS, T-PLL-CDC-STRESS	CHK-03, A-03	COV-04, COV-05	[[OPEN]
F04 (Memória & Ownership)	ST-MEM-01, ST-MEM-02; T-MEM-ISOLATION, T-AES-MULTI-MODE	CHK-01, CHK-02, A-01	COV-06	[OPEN]
F05 (Datapath AES In-Place)	ST-AES-01, ST-AES-04; T-AES-MULTI-MODE, T-AES-ROUNDTRIP, T-CTR-ROLLOVER	CHK-01, CHK-02	COV-01, COV-03, COV-07	[OPEN]
F06 (Multi-bloco & Status)	ST-AES-02; T-AES-MULTI-MODE, T-SNAPSHOT-BUSY	CHK-01, CHK-05, A-04, A-05	COV-02, COV-03	[OPEN]
F07 (Detecção de Erros)	ST-AES-03; T-BOUNDARY-ERR	CHK-02, CHK-04, A-06	COV-01, COV-02	[OPEN]
F08 (Robustez & Runtime)	ST-RST-02, ST-PLL-02, ST-SPI-03, ST-AES-04; T-SNAPSHOT-BUSY, T-RST-RUNTIME	CHK-01, CHK-02, CHK-03, A-03	COV-08	[OPEN]

9.1 Critérios mínimos de fechamento

- Nenhum mismatch ou assertion failure sem explicação na regressão final.
- Todos os itens do VPLAN possuem status final e evidência correspondente.
- Coverage funcional planejado fechado para bins válidos; waivers documentados quando necessários.
- Code coverage analisado e holes relevantes tratados ou justificados.
- Regressão reproduzível com comandos, seeds e versão/tag registrados.
- Verification Guide e Verification Report consistentes com o estado final do ambiente.

10 Referências

[1] RTL e especificação do sistema de topo fornecidos pelo instrutor.

[2] NIST FIPS 197 – Advanced Encryption Standard (AES).

[3] SystemVerilog Language Reference Manual (LRM).

[4] UVM documentation / simulator documentation adotada no projeto.